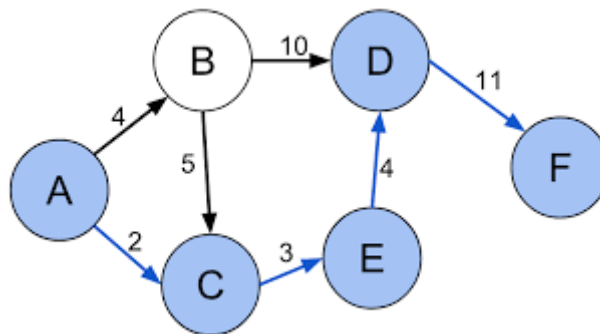




Structures De Données : Projet cheminement optimal dans les graphes

enslIE - FISA 1A



Alistair Baco

Implémentation.....	1
Structure du projet.....	1
Structures de données.....	1
Algorithmes.....	2
Tests.....	2
Résultats obtenus.....	3
a280 p=6%.....	3
att48 p=15%.....	4
att48 p=12%.....	5
Conclusion.....	5

Implémentation

Structure du projet

Le projet est composé de 2 dossiers :

- tsplib_instances, qui, comme son nom l'indique contient les instances de la TSPLIB sur lesquelles les algorithmes sont testés
- src, le code source du projet. Ce dossier est divisé en 3 sous-dossiers (des packages dans le cadre de Java):
 - Algorithms : Contient l'implémentation des algorithmes demandées par le sujet et un fichier d'utilitaire à ces algorithmes
 - Entities : Contient les structures de données utilisées dans le projet (Tas, Graphe...)
 - Tests : Dossier contenant des fichiers testant les différents aspects du projet
 - Enfin il y a un fichier Main.java, contenant un simple hello world. Vous pouvez modifier ce fichier pour essayer le code

Structures de données

Le projet utilise principalement, comme demandé dans le sujet, des Tas et des Graphes. Concrétisés par les classes Graphe.java et Tas.java. La classe Graphe est fortement inspirée de l'implémentation réalisée en cours tandis que la classe Tas a été réalisée à l'aide de [la page wikipedia sur les tas binaires](#).

Il existe néanmoins d'autres structures de données dans ce projet.

Premièrement, la classe TasPointeur est présente dans le dossier Entities bien que non utilisée, c'est une trace de ma première implémentation d'un tas, avant de prendre en compte le critère de performance. J'ai décidé de laisser cette classe dans le projet pour garder la porte ouverte à des tests de différence de performance dans un futur potentiel.

Deuxièmement, il existe une spécialisation de la classe Graphe sous la forme d'une classe GrapheTSPLIB, ajoutant des coordonnées à un sommet de Graphe. Cette classe est nécessaire à l'utilisation de la TSPLIB puisque les sommets y sont localisés par des coordonnées à partir desquelles on crée des arcs.

Enfin, pour implémenter plus simplement les coordonnées des graphes TSPLIB, j'ai ajouté une classe Point2D au projet, diminuant le nombre de paramètres dans les différentes listes mais surtout permettant de contenir la logique pour la distance euclidienne entre deux points.

Algorithmes

Chaque algorithme a été implémenté au plus proche possible de l'algorithme indiqué dans le sujet donné. Avant chaque implémentation d'algorithme, j'ai écrit en commentaire au dessus ce que je comprenais du sujet et des instructions pour moi même pas à pas. Je pense donc qu'il serait redondant de les décrire ici, je vous invite donc à consulter les commentaires de `src/Algorithms/Algorithms.java` pour plus de détails.

Tests

Des tests sur certaines instances de la TSPLIB étaient imposés dans le projet. J'ai donc implémenté ces derniers pour chaque algorithme, avec mesure du temps d'exécution et du nombre de sommets explorés dont vous trouverez les conclusions plus bas. Un point qui m'a temporairement posé problème sur ces tests est la vérification de la véracité d'un plus court chemin trouvé par un algorithme. En effet, d'après ma compréhension de la documentation, la TSPLIB fournit un circuit hamiltonien pour la plupart de ses instances sous forme d'un fichier `.opt.tour`, or elle ne fournit pas de plus court chemin pour chaque valeur de p (ce qui est compréhensible). Pour vérifier le résultat de mes algorithmes, j'ai alors décidé de comparer leur résultat à celui des deux autres. Si les trois trouvent le même résultat, je pars du principe qu'il est correct.

Ensuite, pour prévenir de dysfonctionnements majeurs dans mon code, j'ai décidé de réaliser des tests sur la classe `Tas`, pour laquelle j'ai parfois eu des doutes sur l'implémentation. Ces tests vérifient donc la robustesse de mon tas vis à vis de différentes situations probables.

Dans la même logique, j'ai réalisé un fichier de tests plus basiques qu'un lancement sur des instances de la TSPLIB pour les 3 algorithmes implémentés, ces tests se retrouvent dans `AlgorithmsTests.java`. Ils se concentrent sur des graphes plus simples mais avec chacun une spécificité, pour vérifier le bon comportement des algorithmes.

Enfin, il est notable que ces tests n'utilisent pas JUnit. J'y ai réfléchi mais ai finalement décidé de ne pas le faire afin de ne pas rajouter de dépendances à un projet simple.

Résultats obtenus

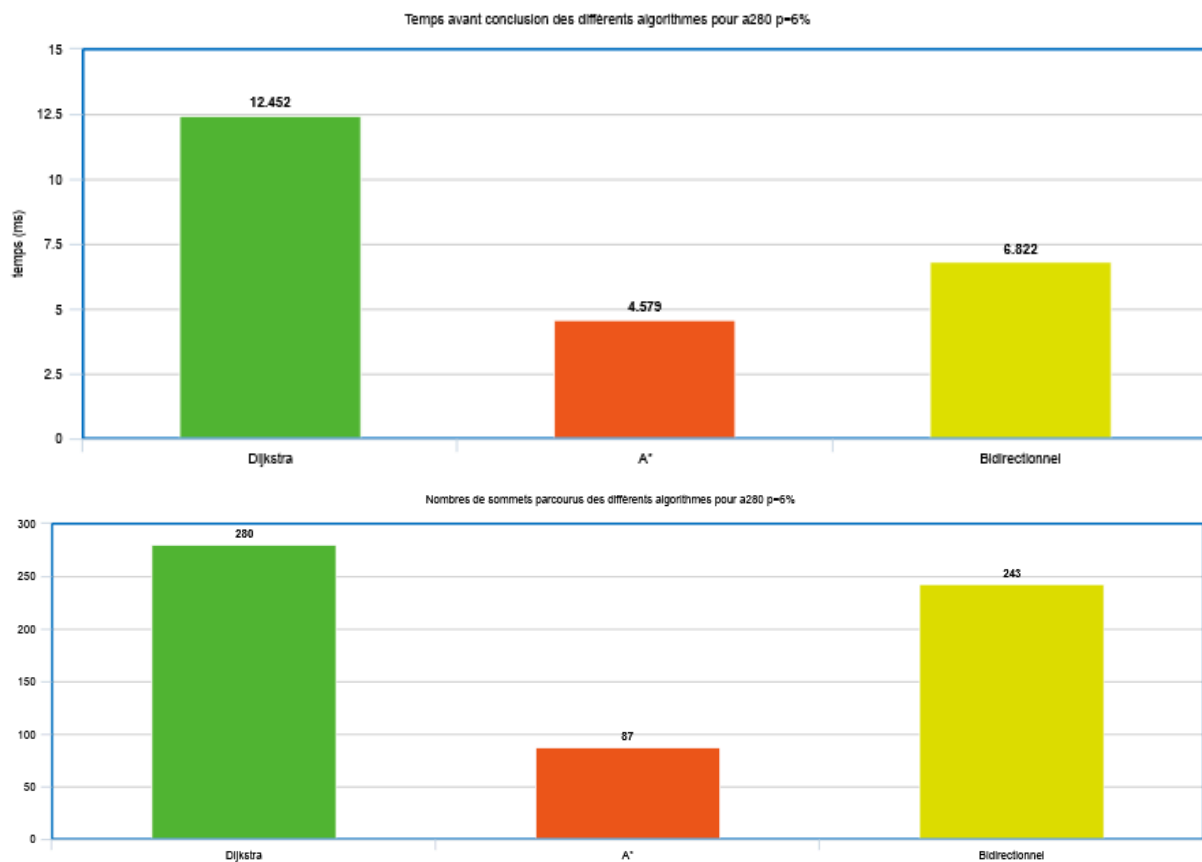
Ci-dessous vous trouverez des graphiques comparant les temps d'exécution et le nombre de sommets parcourus des trois algorithmes sur les 3 instances demandées (a280 6%, att48 15%, att48 12%) dans le sujet.

Les durées indiquées sont en millisecondes (ms) et chaque mesure est une moyenne de 10 exécutions de mon jeu de test.

Ces tests ont été réalisés avec pour CPU un Intel Celeron N4100 @2.4Ghz. Un ordinateur plus récent aura très probablement de meilleures performances côté temps d'exécution.

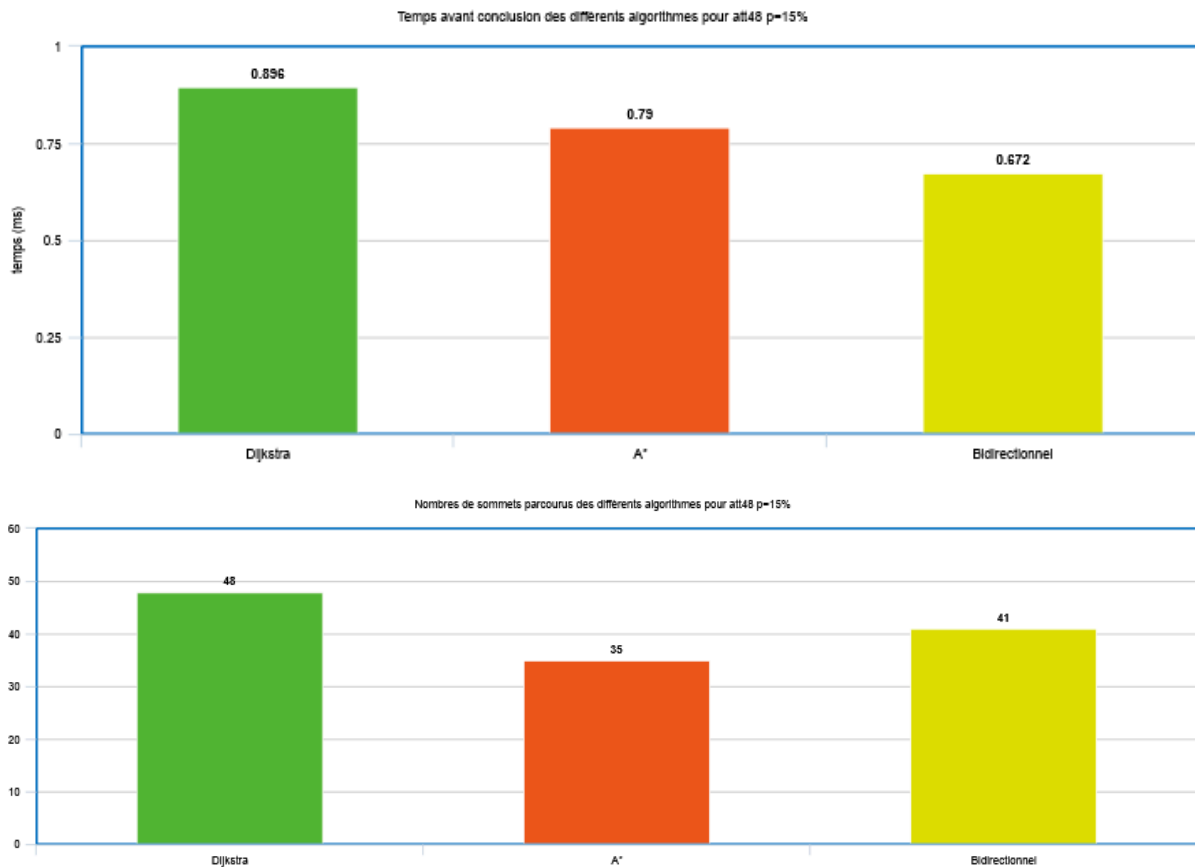
a280 p=6%

L'instance a280 contenant un très large nombre de sommets, il paraît normal que A* soit le plus rapide dans la recherche de plus court chemin, étant le seul avec une volonté d'optimiser le choix de ses sommets explorés.



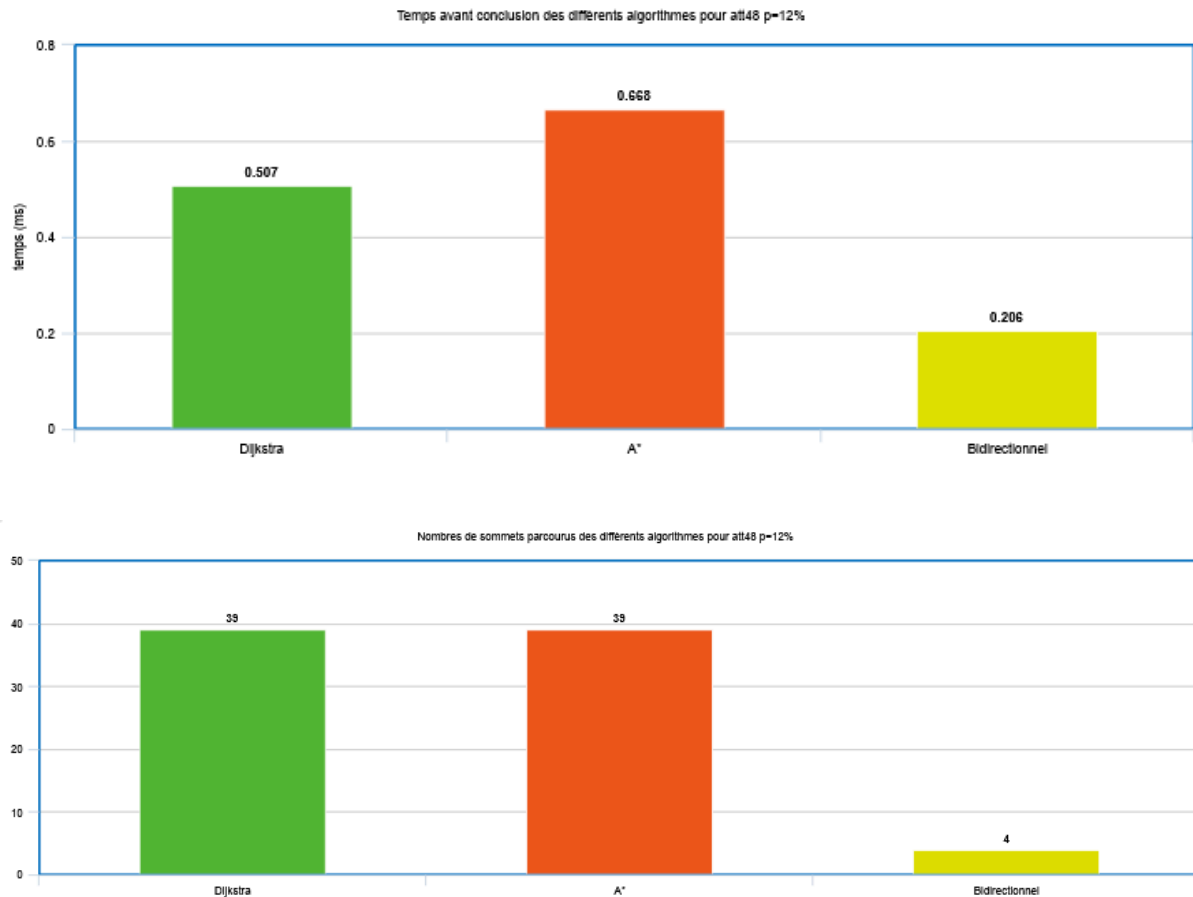
att48 p=15%

Dans un cadre où chaque sommet lié par un arc est plutôt proche l'un de l'autre on peut s'attendre à ce que les optimisations de recherche de A* et de l'algorithme bidirectionnel aient un avantage moindre relativement à Dijkstra. C'est en effet ce qu'on observe empiriquement. Malgré une vitesse d'exécution peu améliorée, A* baisse significativement le nombre de sommets explorés, même dans un cas où les sommets sont proches les uns des autres.



att48 p=12%

Il est notable que ce test conclut qu'il n'existe pas de chemin entre les deux sommets indiqués, il mesure donc la vitesse à laquelle chaque algorithme indique l'absence de chemin. Partant des deux sommets, il paraît naturel que l'algorithme bidirectionnel gagne largement dans ce domaine.



On observe le comportement qu'on attendrait d'A* dans un cadre où il n'existe pas de chemin. Étant un Dijkstra optimisé par son choix d'arête, s'il ne trouve pas de chemin, il explorera toutes les arêtes que Dijkstra aurait exploré.

Conclusion

Sur le faible nombre de situations étudiées avec ces algorithmes, il me semble qu'il est possible de conclure sur deux points :

- L'algorithme A* brille dans les graphes avec beaucoup de sommets, où la direction que prend l'exploration impactera fortement le temps d'exécution
- L'algorithme bidirectionnel semble être l'algorithme le plus flexible des trois, il serait à utiliser dans le cas où on testerait une instance à l'aveugle.

Il serait intéressant par la suite de mener des tests sur les ressources utilisées par ces différents algorithmes afin de voir si le gain de temps d'exécution peut perdre en intérêt dans un cadre où on aurait un ordinateur dont les capacités sont limitées.